

Föreläsning 9

Ändrad 22 Sep 01:08

Tecken representeras av olika heltal i olika system.

Det finns en gemensam grund -- *ASCII* (American Standard Code for Information Interchange). Men ASCII innehåller bara 128 tecken (och knappt det). Där saknas exempelvis bland annat våra svenska tecken.

Heltalen 0 till 127 kan lagras i sju bitar. En byte -- en *char* -- består av åtta bitar. Det finns alltså plats över för ytterligare tecken i en *char* -- utöver ASCII. Med en extra bit dubblar vi antalet möjliga tecken. Åtta bitar ger oss tillgång till heltalen 128--255 för att lagra tecken.

Tyvärr har det skapats flera *olika* konventioner för hur tecknen ska placeras ut bland heltalen 128--255.

Detta ger problem när datorer med olika åttabitarskonventioner ska kommunicera med varandra: exakt vilket heltal är det egentligen som ska motsvara "ö"?

På WWW (World Wide Web) använder man sig av åttabitarskonventionen Latin-1. Men för säkerhets skull finns också en sju-bitarskonvention. Bland annat därför att vissa maskiner inte kan/vill hantera åttabitarstecken -- den åttonde biten ignoreras, eller används till annat(!)

Sju-bitarskonventionen trollar bort alla tal mellan 128 och 255. I sju-bitarskonventionen skrivs exempelvis ö som *ö*; -- en sekvens vanliga ASCII-tecken!

Följande program byter ut åttabitarstecken mot sekvenser av ASCII. Enligt WWW-konventionen. Här är en utskrift av en körning:

```

1  »Å i åa ä e ö«
2  &raquo;&aring; i &aring;a &auml; e &ouml;&laquo;
3  © Gustaf Fröding(?)
4  &copy; Gustaf Fr&ouml;ding(?)
5  ^d

```

Huvudfunktionen *main* består av en *while*-sats som läser ett tecken i taget från användarens inmatningar.

Själva omvandlingen har jag stoppat in i en funktion som jag kallat *printASCII()*. Funktionen returnerar inget värde. Detta anges med nyckelordet *void*.

```

1  /*
2  Övnings exempel 11, ping 1997.
3  Jan Tångring, Umeå 13/9 1997.
4
5  Ersätt icke-ASCII med htmls 7-bitars teckenkoder.
6  Läser från stdin och skriver på stdout.
7
8  Nyheter:
9      - switch
10     - getchar
11     - tecken som int
12     - EOF
13     - villkor med sidoeffekt
14     - funktion utan returvärde
15  */
16

```

```

17 #include <stdio.h>
18
19 #define LAQUO    "&laquo;"
20 #define RAQUO    "&raquo;"
21
22
23 void printASCII(int ch)
24 {
25     switch(ch) {
26         case 0345 : /* å */
27             printf("&aring;");
28             break;
29         case 0344 : /* ä */
30             printf("&auml;");
31             break;
32         case 0366 : /* ö */
33             printf("&ouml;");
34             break;
35         case 0305 : /* Å */
36             printf("&Aring;");
37             break;
38         case 0304 : /* Ä */
39             printf("&Auml;");
40             break;
41         case 0326 : /* Ö */
42             printf("&Ouml;");
43             break;
44         case 0351 /* é */ :
45             printf("&eacute;");
46             break;
47         case 0311 : /* É */
48             printf("&Eacute;");
49             break;
50         case 0255 : /* ¢ */
51             printf("&shy;");
52             break;
53         case 0340 : /* à */
54             printf("&agrave;");
55             break;
56         case 0341 : /* á */
57             printf("&aacute;");
58             break;
59         case 0273 : /* » */
60             printf(RAQUO);
61             break;
62         case 0253 : /* « */
63             printf(LAQUO);
64             break;
65         case 0262 : /* (citationstecken) */
66             printf(RAQUO);
67             break;
68         case 0260 : /* ° */
69             printf("&deg;");
70             break;
71         case 0256 : /* ® */
72             printf("&reg;");
73             break;
74         case 0251 : /* © */
75             printf("&copy;");
76             break;
77         case 0240 : /* (non-breaking space) */
78             printf("&nbsp;");
79             break;
80         default :
81             printf("%c", ch);
82             break;
83     }
84 }
85
86
87 void main()
88 {
89     int ch;
90
91     while ((ch = getchar()) != EOF)
92         printASCII(ch);
93 }

```

Just när du trodde du förstätt allt om tecken i C deklarerar jag plötsligt tecknet `ch` som en `int`! Dessutom ser `while`-villkoret mer än lovligt knökigt ut.

Hemskt ledsen. Men det är inte jag som designat språket!

getchar() returnerar tecken som int

Jag använder `getchar()` i stället för `scanf` när jag läser tecken. Funktionen `getchar()` returnerar tecknet som lästs. Men den returnerar tecknet som en `int`. Inte som en `char`.

Kopplingen mellan heltal och tecken är tack och lov densamma som vanligt! 'A' är fortfarande 65.

Anledningen till att `getchar()` returnerar en `int` är att `getchar()` vill kunna rapportera att något gått snett. Och det vill `getchar()` göra genom att returnera ett negativt tal.

Vi behöver inte veta vilket det negativa talet är. Troligen är det `-1`. Men i vilket fall som helst finns ett färdigt makro för ändamålet, `EOF`. Vilket heltal det än är, så har `EOF` detta värde.

EOF, [Control]+[d]

Alltså: `getchar()` returnerar `EOF` när nåt gått snett. `EOF` är negativt. Därför måste `ch` vara en `int` för att kunna ta emot värdet ordentligt.

Och det som gått snett är att användaren inte har några fler tecken att mata in. Det finns inga fler tecken för `getchar()` att läsa.

Detta har användaren markerat genom att trycka `[Control]+[d]` på tangentbordet. (`[Control]+[d]` är vad som gäller på mitt system. Något annat kanske gäller på ert system).

Villkor med sidoeffekt

Låt oss nu titta på hela `while`-villkoret. Kom ihåg att tilldelningssatsen har ett värde.

I `while`-villkoret händer följande: `ch` sätts till nästa tecken. Eller möjligen `EOF`.

Detta tecken (alternativt `EOF`) blir värdet på tilldelningssatsen. Som jämförs (`!=`) med `EOF`.

Om värdet är `EOF` bryts loopen, och därmed hela programmet.

`while`-loopen fortsätter alltså tills användaren trycker `[Control]+[d]`

OBS! Användaren måste trycka på `[Return]` omedelbart innan `[Control]+[d]` för att datorn ska se `[Control]+[d]`

switch

En `switch`-sats är ofta ett lämpligt alternativ till en `else-if`-stege. Syntaxen är rent gräslig. Men det är bara att vänja sig.

■ Rad 25 `switch(ch)` talar om att det är värdet på `ch` som

bestämmer vad som ska göras.

- Nyckelordet `case` följs av ett heltalsvärde. Om `ch` har detta värde, utförs motsvarande gren: alla satser fram till nästa `break`.
- Jag använder oktala heltalskonstanter. Ingen särskild orsak till det. Jag råkade ha tillgång till värdena som oktala tal.
- Rad 80 om inget `case` matchar, utförs satserna efter nyckelordet `default`.
- Kommentarer finns inte för datorn utan för läsaren.

! Glöm inte att avsluta varje `case` med ett `break`!
Om du glömmer ett `break` kommer datorn att rinna vidare och utföra satser fram till nästa `break`. Oavsett om `switch`-värdet matchar. !

PS Exemplet är en lätt förenklad version av ett program som jag använder just nu till att producera dessa nätsidor.

Notera att versionen inte tar hand om *alla* åttabitarstecken. Programmet varnar heller inte för åttabitarstecken utan `case`. Det borde programmet göra *DS*.

Nästa lektion

